

SemNav

Semantic-Aware Autonomous Navigation with YOLO, ONNX Runtime, LiDAR and Nav2 on ROS 2

Purpose of this document: a complete architecture reference you can hand to Claude Code and build from in roughly 7-10 working days, while ending up with something that reads as real robotics engineering rather than a tutorial replay.

One-line pitch: a mobile robot that detects objects with a camera, localises them with LiDAR, and feeds them into the Nav2 costmap through a custom C++ costmap-layer plugin, so the robot plans around *what* things are, not just *where* there is something solid. A C++ safety gate sits between Nav2 and the wheels.

1. Why this project is a good pick

- **It touches the whole stack:** perception, sensor fusion, localisation, planning, control, safety, tooling. Interviewers can probe any layer and you have an answer.
- **The differentiator is small but real:** a custom `nav2_costmap_2d` plugin plus **camera-to-LiDAR bearing fusion**. Most student projects only run stock Nav2 or only run YOLO. Joining the two is the part that shows depth.
- **Everything is measurable:** inference latency, localisation error against Gazebo ground truth, navigation success with and without the semantic layer. Numbers on a README beat adjectives.
- **Scope is controlled:** no warehouse, no business logic, no manipulation. Everything lives in simulation, so it is reproducible and demoable from a laptop.
- **Honest caveat:** I do not know xboom's exact focus areas. Check their site and tweak the demo framing (for example, emphasise latency if they do embedded work, or the safety gate if they build AMRs).

2. Requirement-to-component mapping

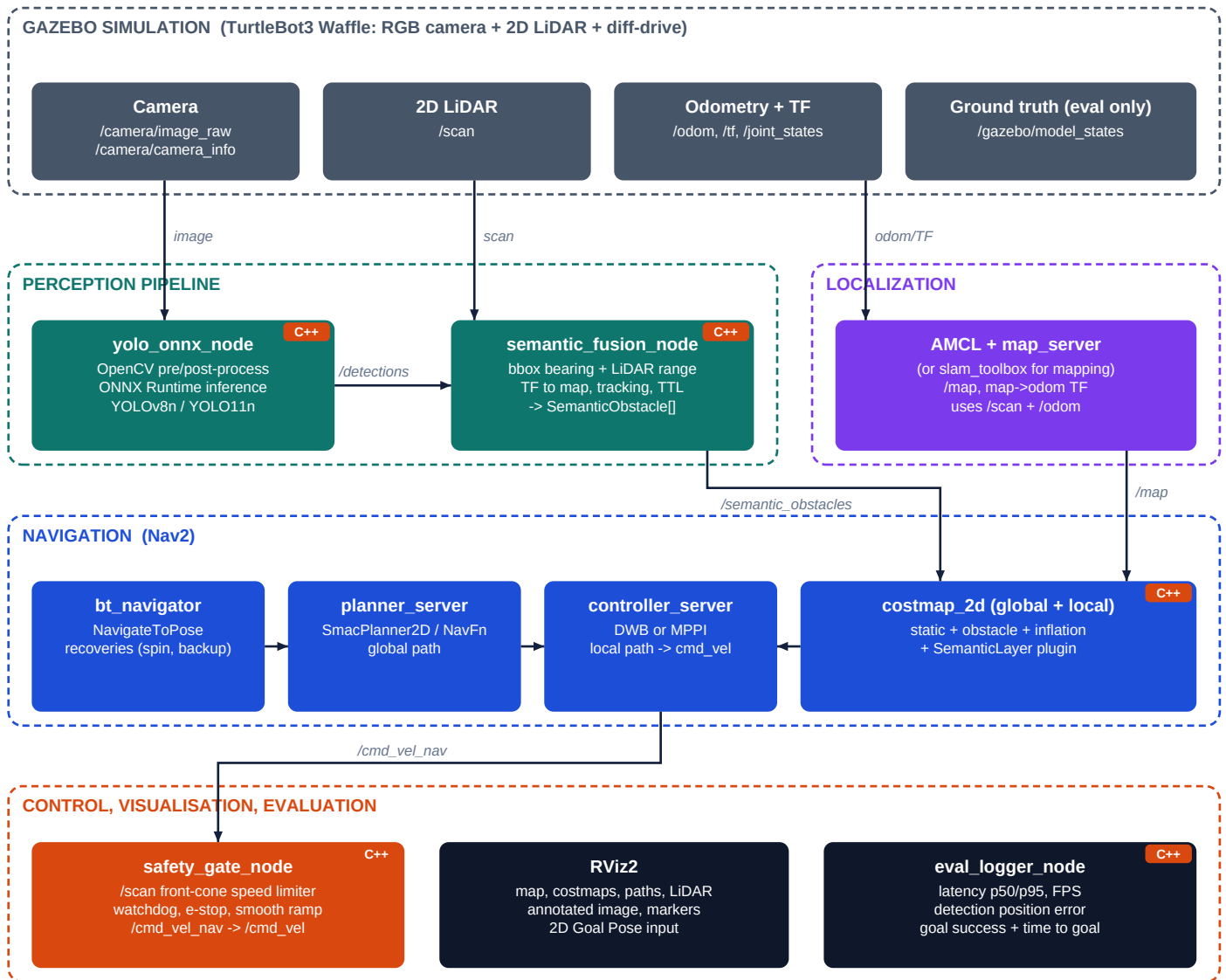
Your requirement	How this design satisfies it
Receive camera and LiDAR data	Gazebo TurtleBot3 Waffle publishes <code>/camera/image_raw</code> , <code>/camera/camera_info</code> , <code>/scan</code> . QoS: sensor-data (best effort, depth 5).
YOLO object detection	YOLOv8n (or YOLO11n) pretrained on COCO, exported once to ONNX. No training needed.
OpenCV image processing	Letterbox resize, BGR-to-RGB, normalise, blob creation, NMS (cv::dnn::NMSBoxes), box drawing, cv_bridge conversions.
ONNX Runtime	C++ Ort::Session in <code>yolo_onnx_node</code> , CPU provider by default, CUDA provider as an optional parameter.
ROS 2 communication	Topics for streams, one custom message, TF2 for frames, parameters for tuning, launch files for composition, lifecycle-managed Nav2.
LiDAR + Nav2 navigation / obstacle avoidance	AMCL localisation, Nav2 planner and controller, LiDAR obstacle layer, plus the semantic layer and safety gate.
C++ for robot / control / navigation nodes	Fusion node, costmap plugin, safety gate, eval logger and the YOLO node are all C++17 (ament_cmake). Launch and RViz config are the only Python/YAML.
Visualise in RViz2	One saved .rviz config: map, both costmaps, paths, LiDAR, annotated camera image, class-coloured 3D markers, TF.
No warehouse / inspection / business logic	Goals come from RViz 2D Goal Pose or a small waypoint list. Nothing domain-specific.

3. Technology choices

Item	Choice	Reason
OS / ROS	Ubuntu 22.04 + ROS 2 Humble (or 24.04 + Jazzy)	Most tutorials, Nav2 docs and TurtleBot3 packages match; Claude Code has seen the most examples for Humble. Pick one and never mix.

Item	Choice	Reason
Simulator	Gazebo with TurtleBot3 Waffle (has camera + 2D LiDAR)	Zero hardware. On Humble use Gazebo Classic 11 with turtlebot3_simulations; on Jazzy use the Gazebo Harmonic variant.
Detector	YOLOv8n or YOLO11n, ONNX, 640x640, opset 12-17	Smallest model, runs real time on laptop CPU. Note: Ultralytics weights are AGPL-3.0; fine for a portfolio, mention it in the README.
Inference	ONNX Runtime C++ (prebuilt release tarball)	Linked via CMake find_package or a vendored path. Keep it in a third_party folder, not apt.
Imaging	OpenCV 4.x, cv_bridge, image_transport	Standard in ROS 2 desktop installs.
Navigation	Nav2 (bt_navigator, SmacPlanner2D or NavFn, DWB or MPPI), AMCL, map_server, slam_toolbox for map creation	Production-grade stack the plugin will extend.
Build / test	colcon, ament_cmake, GoogleTest, launch_testing	Unit tests on pure functions (letterbox, NMS, projection) are cheap and impressive.
Tooling	Claude Code, git, clang-format, GitHub Actions (build + unit tests)	CI badge on the README signals engineering habits.

4. System architecture



Reading the diagram: orange "C++" tags mark the nodes you write. Blue is stock Nav2 configured by you. Purple is stock localisation. Detections flow top-left to the costmap; the LiDAR is used twice (by Nav2's obstacle layer and by the fusion node), which is deliberate: LiDAR stays the authority on geometry, the camera only adds meaning.

Data flow in six steps

- Camera frame arrives; yolo_onnx_node keeps only the newest frame, runs ONNX inference, publishes vision_msgs/Detection2DArray and an annotated image.
- semantic_fusion_node converts each bounding box to a bearing using camera intrinsics and the camera-to-laser TF, then reads the matching range from the latest /scan.
- The resulting point is transformed into the map frame, tracked across frames (nearest-neighbour + smoothing + time-to-live) and published as SemanticObstacleArray.
- SemanticLayer, a plugin inside both Nav2 costmaps, paints class-specific cost discs (for example a wide, high-cost bubble around a person, a small one around a chair).
- Nav2 plans and controls against the combined costmap and publishes /cmd_vel_nav.
- safety_gate_node limits or zeroes the command based on live LiDAR distance and publishes the final /cmd_vel to the robot.

5. Workspace and package layout

```
semnav_ws/
  CLAUDE.md          <- project rules for Claude Code (see section 13)
  README.md docs/    <- architecture diagram, demo GIF, results table
  models/            <- yolov8n.onnx, coco.names (git-lfs or download script)
  third_party/onnxruntime/ <- prebuilt ORT, fetched by scripts/setup_ort.sh
  scripts/           <- setup_ort.sh, export_yolo.py, run_eval.sh
  src/
    semnav_msgs/      <- SemanticObstacle.msg, SemanticObstacleArray.msg
    semnav_perception/ <- yolo_onnx_node, semantic_fusion_node (C++)
      include/semnav_perception/{yolo_detector.hpp, letterbox.hpp, fusion.hpp, tracker.hpp}
      src/ test/       <- GoogleTest for letterbox, NMS decode, projection
    semnav_costmap/    <- SemanticLayer (nav2_costmap_2d plugin), plugin.xml
    semnav_control/    <- safety_gate_node (C++)
    semnav_eval/       <- eval_logger_node, scenario runner
    semnav_bringup/    <- launch/, config/nav2_params.yaml, maps/, worlds/, rviz/semnav.rviz
```

Keeping semnav_msgs in its own package is the ROS 2 convention and avoids circular build dependencies.

6. Interfaces: topics, messages, frames

Topic	Type	Producer -> Consumer	QoS / note
/camera/image_raw	sensor_msgs/Image	Gazebo -> yolo_onnx_node	SensorData (best effort), depth 1-2
/camera/camera_info	sensor_msgs/CameraInfo	Gazebo -> fusion	Read once, cache fx, cx
/scan	sensor_msgs/LaserScan	Gazebo -> AMCL, Nav2, fusion, safety	SensorData, depth 5
/detections	vision_msgs/Detection2DArray	yolo -> fusion	Reliable, depth 5. Header stamp copied from the image.
/detections/image	sensor_msgs/Image	yolo -> RViz	Annotated frame, best effort
/semantic_obstacles	semnav_msgs/SemanticObstacleArray	fusion -> SemanticLayer, markers	Reliable, depth 5. Frame = map
/semantic_markers	visualization_msgs/MarkerArray	fusion -> RViz	Cylinder + text per obstacle, colour by class
/cmd_vel_nav	geometry_msgs/Twist	Nav2 controller -> safety gate	Remap controller output
/cmd_vel	geometry_msgs/Twist	safety gate -> robot	Only publisher to the robot
/metrics	std_msgs/Float32MultiArray or diagnostic_msgs	yolo, eval -> logger	Latency, FPS

Custom messages

```
# SemanticObstacle.msg
std_msgs/Header header      # frame_id = "map", stamp = source image stamp
uint32 id                  # stable track id
string class_name           # "person", "chair", ...
float32 confidence
geometry_msgs/Point position # in header.frame_id
float32 range               # LiDAR range used (m), for debugging
float32 radius              # footprint radius (m), from per-class table
builtin_interfaces/Duration ttl

# SemanticObstacleArray.msg
std_msgs/Header header
semnav_msgs/SemanticObstacle[] obstacles
```

TF tree

map -> odom (AMCL) -> base_footprint -> base_link -> {base_scan (LiDAR), camera_link -> camera_rgb_optical_frame}
The fusion node never hard-codes extrinsics; it asks TF2 for camera_optical -> base_scan at the image timestamp.

7. Node specifications

7.1 yolo_onnx_node (C++, the performance-critical node)

Aspect	Design
Pipeline	Letterbox to 640x640 (keep aspect, pad grey 114) -> BGR to RGB -> float32 /255 -> NCHW tensor -> Ort::Session::Run -> decode -> NMS -> undo letterbox -> publish.
Decode	YOLOv8 output is [1, 84, 8400]: 4 box values (cx, cy, w, h) + 80 class scores per anchor, no objectness. Transpose, take max class score, filter by conf_threshold (0.35), then cv::dnn::NMSBoxes (IoU 0.45).
Concurrency	Subscriber callback only stores the newest cv::Mat under a mutex and signals a condition variable. A worker thread runs inference. Old frames are dropped instead of queued, so latency never grows.
ORT options	intra_op_num_threads (param, default 4), GraphOptimizationLevel ORT_ENABLE_ALL, pre-allocated input buffer reused every frame, optional CUDA execution provider via param use_cuda.
Parameters	model_path, conf_threshold, iou_threshold, input_size, class_filter (list of COCO names to keep, e.g. person, chair, bottle), use_cuda, publish_annotated.
Metrics	Time each stage (preprocess, infer, postprocess) with std::chrono, log rolling p50/p95 every 5 s, publish on /metrics.
Tests	GoogleTest: letterbox round-trip maps a known box back to within 1 px; decode on a synthetic tensor returns the planted box; NMS removes a duplicate.

7.2 semantic_fusion_node (C++, the clever bit)

Aspect	Design
Why bearing-range	The Waffle has a 2D LiDAR, so full pixel-to-point projection is impossible. Instead use the bbox's horizontal span as an angular sector and read ranges from that sector. It is simple, robust and easy to explain.
Algorithm	1) For bbox [u_min, u_max] build rays $r = ((u-cx)/fx, 0, 1)$ in the optical frame. 2) Rotate rays into the laser frame with TF at the image stamp and get yaw angles. 3) Collect scan ranges inside the central 60% of that sector. 4) Reject invalid / out-of-range values, take the median (robust to wall pixels leaking in). 5) Position = range * (cos yaw, sin yaw), pad by class radius. 6) Transform to map.
Time sync	Look up TF at the detection stamp, and pick the scan closest in time (cache last N scans in a ring buffer). Use message_filters ApproximateTime if you prefer, with slop 0.1 s.
Tracking	Nearest-neighbour association by class and distance gate (0.6 m), exponential smoothing of position, id assignment, miss counter; obstacle expires after ttl (default 2 s) so ghosts vanish.
Outputs	SemanticObstacleArray plus MarkerArray (cylinder radius from class table, colour by class, text label with confidence).
Parameters	class_radius map (person 0.35, chair 0.25, default 0.3), min/max_range, sector_fraction, assoc_gate, ttl, smoothing_alpha.
Known limit	Objects not at the LiDAR height (a table top) can be missed or mis-ranged. State this in the README; it shows you understand the model.

7.3 SemanticLayer (C++ nav2_costmap_2d plugin)

Aspect	Design
Base class	nav2_costmap_2d::Layer, exported with PLUGINLIB_EXPORT_CLASS and a plugin.xml; added to both global and local costmaps in nav2_params.yaml.
Lifecycle	onInitialize: declare params, subscribe /semantic_obstacles. updateBounds: expand the dirty window around each live obstacle. updateCosts: paint into the master grid. reset(): clear cache.
Cost model	Per class: core radius = LETHAL, then exponential decay out to class_inflation (person 1.0 m, chair 0.3 m, default 0.2 m) using $\text{cost} = 252 * \exp(-k * (d-r))$. Combine with std::max so you never lower an existing cost.
Thread safety	Subscription thread writes into a snapshot under a mutex; costmap thread copies it at the start of updateBounds.
Frames	Obstacles are in map; use tf_ to convert to the costmap's global frame if rolling window is used.
Why it matters	Without it, a person and a trash can look identical to the planner. With it, the robot leaves a wider berth around people. That is a concrete, testable behaviour.

7.4 safety_gate_node (C++, control)

Aspect	Design
Inputs / output	Subscribes /cmd_vel_nav and /scan; publishes /cmd_vel at a fixed 20 Hz timer, never directly from the callback.
Logic	Minimum range in a forward cone (default +/-30 deg, widened with angular speed). Scale factor $s = \text{clamp}((d - d_{\text{stop}}) / (d_{\text{slow}} - d_{\text{stop}}), 0, 1)$. Output $v = s * v_{\text{cmd}}$; angular velocity is kept so the robot can still turn away.

Aspect	Design
Watchdog	If no /cmd_vel_nav for 0.5 s or no /scan for 0.3 s, publish zero. Acceleration limiter (max delta v per tick) avoids jerk.
Be ready to explain	How it differs from nav2_collision_monitor (Nav2 ships one). Honest answer: this one is a deliberately small custom implementation to show control-layer reasoning; in production you would evaluate the stock package first.
Tests	Feed a synthetic scan with a wall at 0.3 m and check output is zero; feed far scan and check passthrough.

7.5 eval_logger_node (C++ or Python, small)

- Logs inference p50/p95 latency and FPS from /metrics.
- Compares fused obstacle positions against Gazebo ground truth to give mean localisation error per class.
- Sends a fixed list of goals through NavigateToPose and records success, time, path length, minimum clearance to people. Writes CSV for the results table.

8. Nav2 configuration decisions

Area	Decision
Localisation	Mode A (default for demos and eval): pre-saved map + AMCL + map_server. Mode B (mapping demo): slam_toolbox online_async to build the map, then save with map_saver.
Global planner	SmacPlanner2D (or NavFn if you want fewer moving parts).
Controller	DWB is the safe default. MPPI is more impressive if time allows; the change is one YAML block.
Costmaps	Global: static, obstacle (LiDAR), semantic, inflation. Local: rolling window 3x3 m, obstacle, semantic, inflation. Keep semantic before inflation.
Topic remap	Controller cmd_vel output remapped to /cmd_vel_nav so only the safety gate talks to the robot.
Recoveries	Default BT with spin, backup, wait. Do not write a custom BT unless you finish early.

9. RViz2 configuration (saved as semnav.rviz)

- Map, global costmap and local costmap (costmap shows the semantic discs as distinct blobs around people).
- LaserScan, global plan, local plan, robot footprint, TF.
- Image display on /detections/image with boxes and labels.
- MarkerArray on /semantic_markers: coloured cylinders with class names above them.
- Tools: 2D Pose Estimate (AMCL init) and 2D Goal Pose (send goals). Load everything with one launch argument so the demo starts in one command.

10. Performance, robustness, and quality targets

Metric	Target (aspirational, adjust to your laptop)	How measured
Inference latency (YOLOv8n, CPU)	p50 under about 40 ms, p95 under about 70 ms	std::chrono per stage, /metrics
Perception throughput	10+ FPS sustained, no queue growth	ros2 topic Hz /detections
Image-to-obstacle latency	Under 150 ms	Header stamp vs publish time
Obstacle position error	Median under 0.3 m in range 1-4 m	Compare with Gazebo ground truth
Navigation success	At least 9 of 10 runs in each scenario	eval_logger CSV
Person clearance	Higher minimum clearance with semantic layer than without	A/B run, same goals, layer on/off
Quality gates	Zero warnings from colcon build with -Wall -Wextra; unit tests pass in CI	GitHub Actions

Important: these are targets, not results. Only put numbers in the README that you actually measured.

11. Risks and mitigations

Risk	Impact	Mitigation
YOLO misses or mislabels simulated objects (COCO is trained on real photos)	Perception demo looks broken	On Day 2 run a quick Python ONNX check against sim frames. Use Gazebo models with realistic textures (person, chair, bottle). Fallback: feed a recorded real video or rosbag to the same node, and show both. Lower conf_threshold; do not fine-tune unless necessary.
ONNX Runtime linking / version issues in CMake	Lost day	Pin one ORT version, fetch via script, document the CMake snippet in CLAUDE.md. Get a hello-world Ort::Session building before writing any ROS code.
Costmap plugin does not load	Lost day	Start from the Nav2 tutorial 'Writing a New Costmap2D Plugin', make it paint one fixed disc first, then wire the subscription.
Nav2 tuning rabbit hole	Time sink	Time-box tuning to half a day. Use the TurtleBot3 Nav2 params as the base and change only what you need.
Simulator CPU load (Gazebo + Nav2 + YOLO)	Laggy demo, timing noise	Use a small world, headless Gazebo for evaluation, reduce camera to 640x480 at 15 Hz.
Claude Code generates code you cannot explain	Fails the interview	Read every file. Ask Claude to explain each module and write short docs/design notes in your own words. Never merge what you do not understand.

12. Build plan (10 days, compressible to 7)

Day	Goal	Done when
1	Install ROS 2, Nav2, TurtleBot3 sim. Create workspace, git repo, CLAUDE.md. Drive robot in Gazebo and see everything in RViz.	Teleop works; camera and scan visible in RViz.
2	Build a map with slam_toolbox and save it. Bring up Nav2 with AMCL; send a goal from RViz. Python ONNX sanity check on sim camera frames.	Robot reaches a goal; YOLO detects at least one object class in sim.
3	semnav_msgs. yolo_onnx_node minimal: subscribe, infer, publish Detection2DArray and annotated image.	Boxes visible in RViz.
4	Latency stages, worker thread with frame dropping, parameters, class filter, GoogleTest for letterbox/decode/NMS.	p50/p95 reported; unit tests green.
5	semantic_fusion_node: intrinsics, TF, bearing-range, tracking, markers.	Markers sit on the right objects in RViz.
6	SemanticLayer plugin: fixed-disc prototype first, then subscription and class costs; add to both costmaps.	Costmap visibly changes around a person.
7	safety_gate_node, topic remap, single bringup launch file for everything.	One command starts the full system.
8	Scenarios and eval_logger: A/B with and without semantic layer, collect CSV, compute table.	Results table with real numbers.
9	RViz config polish, README, architecture figure, 60-90 second demo video / GIF, CI workflow.	A stranger can clone and run it in under 15 minutes.
10	Buffer, bug fixes, optional stretch goal, interview rehearsal.	Repo tagged v1.0.

If you fall behind, cut in this order: (1) MPPI, (2) tracking smoothing, (3) CUDA, (4) eval automation (run scenarios manually, log by hand). **Never cut:** the SemanticLayer plugin, the unit tests, the latency numbers, the demo video.

13. Working with Claude Code effectively

CLAUDE.md contents (paste at repo root)

```
# SemNav project rules
- ROS 2 <distro>, Ubuntu <ver>, C++17, ament_cmake. No Python nodes except launch/scripts.
- Build: colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release
- Test: colcon test --event-handlers console_direct+
- Format: clang-format (Google style, 100 cols). Warnings: -Wall -Wextra -Wpedantic, treat as errors.
- Use rclcpp parameters for all tunables. No magic numbers in code.
- Keep pure logic (letterbox, decode, NMS, projection, cost math) in header-only classes with no ROS dependency so they are unit-testable.
- After every change: build, run relevant tests, and show me the command output.
- ONNX Runtime lives in third_party/onnxruntime (see scripts/setup_ort.sh).
- Never add dependencies without asking. Small commits with clear messages.
- Architecture source of truth: docs/SemNav_Architecture.pdf
```

Prompt pattern per phase

- **Plan first:** "Read CLAUDE.md and the architecture doc. Propose a step plan for yolo_onnx_node. Do not write code yet."
- **Small vertical slices:** one node, one behaviour, one test per prompt. "Implement only letterbox() with a unit test."
- **Verify with ROS tools:** ask Claude to run `ros2 topic hz`, `ros2 topic echo --once`, `ros2 run tf2_tools view_frames` and report what it sees.
- **Understand it:** "Explain this file as if I must defend it in an interview, and list three ways it could fail."
- **Debug with evidence:** paste the actual error and the launch log, not a paraphrase.

Example first prompt

Read CLAUDE.md. Create package `semnav_perception` (`ament_cmake`, C++17) with a header-only `YoloDetector` class: load an ONNX model via ONNX Runtime C++, run inference on a `cv::Mat`, return `std::vector<Detection{cv::Rect2f box; int class_id; float score}>`. Implement `letterbox`, YOLOv8 [1,84,8400] `decode` and `NMS`. No ROS code yet. Add `GoogleTest` tests for `letterbox` round-trip and `decode` on a synthetic tensor. Provide a tiny CLI main that runs on an image file and saves the annotated result. Build and run the tests and show me the output.

14. Making it land with xboom

Repository checklist

- README top: 10-second GIF of the robot avoiding a detected person, then a one-paragraph problem statement.
- Architecture diagram (this one, cleaned), results table with real measured numbers, 'Limitations' section.
- One-command setup script and one-command demo launch. CI badge. Tagged release.
- docs/design-notes.md written in your own words: why bearing-range fusion, why a plugin, why frame dropping.

Resume bullets (replace brackets with your measured values)

- Built a ROS 2 (C++) navigation stack that fuses YOLOv8 detections (ONNX Runtime, OpenCV) with 2D LiDAR to create class-aware obstacles in Nav2.
- Wrote a custom nav2_costmap_2d plugin and a C++ safety gate; achieved [X] ms p50 inference latency and [Y]% navigation success over [N] runs in Gazebo.
- Quantified perception accuracy against simulator ground truth (median error [Z] m) and automated A/B evaluation of the semantic layer.

Questions to rehearse

- Why fuse by bearing and range instead of projecting LiDAR points into the image?
- What happens when YOLO and LiDAR disagree, or when the detection is stale?
- Why drop frames rather than queue them? What QoS did you choose and why?
- How does your costmap layer avoid data races, and how do you clear ghost obstacles?
- How does your safety gate differ from nav2_collision_monitor?
- What would change on a real robot (calibration, time sync, 3D LiDAR, TensorRT, real-time executors)?

15. Stretch goals (only after v1.0)

- Velocity estimation of tracked people and a predictive cost cone in front of them.
- TensorRT or ONNX Runtime CUDA execution provider benchmark chart (CPU vs GPU).
- Composable node container with intra-process communication for zero-copy images.
- Run the same perception node on a real webcam or recorded real-world rosbag.
- Docker image plus devcontainer so reviewers can run the project without installing ROS.